Experimental Perfomance of Bilipschitz Scalars-Final Report

Yi Lu

April 2025

1 Abstract

In our project, we aim to conduct several experiments to test the performance of bilipschitz machine learning and compared it with the model without bilipschitz property. Based on the problem setting of springy double pendulum and current achieved model performance thanks to Yao(2021, et al), we aimed to consider a model with bilipschitz property and tested whether it can achieve better model performance.

2 Introduction

2.1 Problem Background Setting

Problem setting: We aim to predict the trajectory of a double pendulum given different initial states. Given a fixed origin o and gravity acceleration g, the first spring with stiffness k_1 has one end fixed at the origin while the other end connected with a ball with mass m_1, and the second spring with stiffness k_2 has one end connected with the same ball with mass m_1 while the other end connected with another ball with mass m_2. The objective of our model is to train a valid model to predict the system status with given time t given the initial system status at time $t = 0$. More specifically, the status of the system includes both variant(momentum and position of both balls) and invariant status(spring stiffness, origin position, gravity acceleration) with respect to time t. We hope to achieve a model whose outputs match the ground truth trajectories well, which were derived from physical experiments. Namely, given initial momentum and position of m_1 and m_2, $z(t_0) = (p_1(t_0), p_2(t_0), q_1(t_0), q_2(t_0)) \in (R^3)^4$ we want to estimate momentum and position of m_1 and m_2 at any time t_j, as $z(t_j) = (p_1(t_j), p_2(t_j), q_1(t_j), q_2(t_j)) \in (R^3)^4$.

2.2 Notation

m_i: mass of ball i, where $i = 1, 2$
k_i: stiffness of spring i, where $i = 1, 2$
$p_i(t) \in R^3$: momentum of ball i at time t
$q_i(t) \in R^3$: position of ball i at time t
$z(t) = (q_1(t), q_2(t), p_1(t), p_2(t)) \in (R^3)^4$: the status of the whole system at time t, more specifically, $z^j(t)$ denote the system status at time t for jth trajectory.

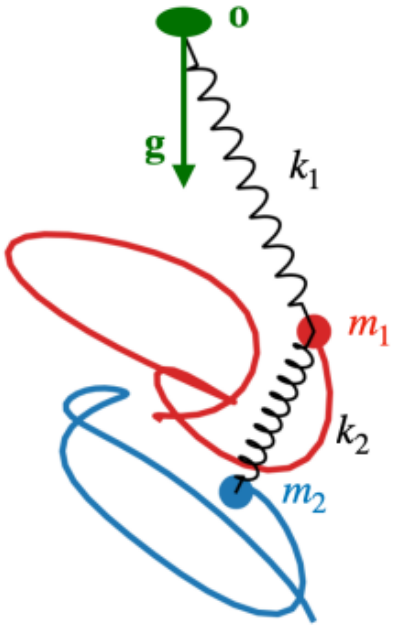

Figure 1: Springy Double Pendulum

$q_o \in R^3$: position of the origin
$g \in R^3$: gravity acceleration
$\theta : R^{d\times n} \to R^{d\times d}$: bilipschitz map for a number of vectors
$\psi : R^d \times R^d \to R$: bilipschitz function for scalar result of vector pair
$f : (R^3)^4 \times R^3 \times R^3 \times R \to (R^3)^4$: trained model to predict dynamics

2.3 System Mechanic Energy

Ignoring the influence of air friction, the mechanic energy of the whole system conserves within a range of time period. In particular, system's energy $\mathcal{E}$ is composed of potential energy $\mathcal{U}$ and kinetic energy $\mathcal{T}$.

Kinetic energy $\mathcal{T}$ is composed of kinetic energy of two balls m_1, m_2, and potential energy $\mathcal{U}$ is composed of elastic potential energy of two springs k_1, k_2 and gravitational potential energy of two balls m_1, m_2.

$$\begin{cases} \mathcal{T} = \frac{|p_1|^2}{2m_1} + \frac{|p_1|^2}{2m_1} \\ \mathcal{U} = \frac{1}{2}k_1(|\mathbf{q}_1 - \mathbf{q}_o| - l_1)^2 + \frac{1}{2}k_1(|\mathbf{q}_1 - \mathbf{q}_o| - l_1)^2 - m_1\mathbf{g}(\mathbf{q}_1 - \mathbf{q}_o) - m_2\mathbf{g}(\mathbf{q}_2 - \mathbf{q}_o) \end{cases}$$

In Hamiltonian Neural Network setting, we utilize the property that the mechanic energy of the whole system conserves.

2.4 Dataset

We are given N sequences of training input, where each of them recorded the status of the system with $t \in \{t_0, t_1, ..., t_T\}$ with $t_0 = 0$ as a trajectory.

In general, the whole training dataset $\mathbf{Z}_{train} = [Z^1_{train}, Z^2_{train}, ..., Z^N_{train}]$ denoting N sample points, where $Z^j_{train} = [z^j_{train}(t_0), z^j_{train}(t_1), ..., z^j_{train}(t_T)]$,
and $z^j_{train}(t_k) = (q^j_{train,1}(t_k), q^j_{train,2}(t_k), p^j_{train,1}(t_k), p^j_{train,2}(t_k)) \in (R^3)^4$ is the system status for j-th trajectory at time t_k. For model training purpose, we will flatten the vector $z^j_{train}(t_k)$ into R^{12}. As a result, the whole dataset is in $R^{N\times(T+1)\times 12}$ if we flatten each $z^j_{train}(t_k)$. As a supervised machine learning model, we can treat $\mathbf{X}_{train} = [Z^1_{train}(t_0), Z^2_{train}(t_0), \cdots, Z^N_{train}(t_0)] \in R^{12\times N}$ as features, and $\mathbf{Y}_{train} = [Z^1_{train}(t_{1:T}), Z^2_{train}(t_{1:T}), \cdots, Z^N_{train}(t_{1:T})] \in R^{12\times T\times N}$ as labels, with $Z^j_{train}(t_{1:T}) = [z^j_{train}(t_1), ..., z^j_{train}(t_T)] \in R^{12\times T}$ as the later trajectory for j-th sample.

After training the model f, we evaluate the performance in test dataset which the model has never seen before. Given $\mathbf{Z}_{test} = [Z^1_{test}, Z^2_{test}, ..., Z^m_{test}]$, extract corresponding features $\mathbf{X}_{test} = [Z^1_{test}(t_0), Z^2_{test}(t_0), \cdots, Z^m_{test}(t_0)] \in R^{12\times m}$,
and predict $\hat{Y}^j_{test} = f(\mathbf{X}_{test}, q_o, g, t_j)$,
$\Rightarrow \hat{\mathbf{Y}}_{test} = [\hat{Z}^1_{test}(t_{1:T}), \hat{Z}^2_{test}(t_{1:T}), \cdots, \hat{Z}^m_{test}(t_{1:T})] \in R^{12\times T\times m}$

Meanwhile, given ground truth labels$\mathbf{Y}_{test} = [Z^1_{test}(t_{1:T}), Z^2_{test}(t_{1:T}), \cdots, Z^m_{test}(t_{1:T})] \in R^{12\times T\times m}$,
and similarly, $Z^j_{test}(t_{1:T}) = [z^j_{test}(t_1), ..., z^j_{test}(t_T)] \in R^{12\times T}$ as the later trajectory for j-th sample, we can conduct model performance evaluation.

2.5 Model

The input vector of the model is $\mathbf{X} = [Z^1(t_0), Z^2(t_0), \cdots, Z^N(t_0)]$, and the output is the function $f : (R^3)^4 \times R^3 \times R^3 \times R \to (R^3)^4$. Considering natural physics law, we are interested in training a model that is equivariant to rotation. In this experiment, we employ Scalar Equivariant Multilayer Perceptron(Scalar EMLP) to construct the model. To train the model, we consider both Neural Ordinary Differential Equations(Neural ODE) and Hamiltonian Neural Networks(HNN).

2.6 Loss function

We consider $l(\theta) = \sum_{i=0}^{N}\sum_{j=0}^{T} ||z^i(t_j) - \hat{z}^i(t_j)||^2$ as the loss function, with $\hat{z}^i(t_j) = f_\theta(z^i(0), q_o, g, t_j)$ as the predicted future trajectory.

3 Experiment and Evaluation Results

3.1 Hyperparameter

The experiment was conducted with the following hyperparameters:
epoch number: 2000
number of data samples: 5000
learning rate: 0.001
neural network structure: 3 layers, with 100 hidden nodes in each layer

None of these parameters are tuned, but it can be considered for future work.

3.2 Original Result

Without the bilipschitz map, the following results are derived for neural ODE and hamiltonian neural network method. The author managed to produce better results than baseline models.

3.2.1 Neural ODE

Train MSE	test MSE	test Rollout	val MSE
0.000008	0.000022	0.009506	0.000032

3.2.2 HNN

Train MSE	test MSE	test Rollout	val MSE
0.000009	0.000021	0.006928	0.00004

3.3 Bilischitz Map Result

After changing the original inner product into bilipschitz map, the evaluation results were recorded below.

3.3.1 Neural ODE

Train MSE	test MSE	test Rollout	val MSE
0.000008	0.000022	0.009237	0.000032

3.3.2 HNN

Train MSE	test MSE	test Rollout	val MSE
0.000031	0.000048	0.011083	0.000091

3.4 Summary

As we can see, the evaluation results are almost the same for Neural ODE method(bilipschitz version got slightly better), but it got much worse for HNN. As a result, this bilipschitz function mapping doesn't help with the model performance.

4 Contribution

To summarize, this project made an initial effort to test bilipschitz function influence for equivariant machine learning model performance. Matrix square root was considered as the map, and evaluation results were compared with those without bilipschitz mapping.

Apart from that, the evaluation results of both Neural ODE and HNN are derived and compared for the new model, and each of them was compared to the one without bilipschitz mapping.

Ultimately, numerical experiments were conducted in terms of lower and upper bound for the bilipschitz map θ.

5 Discussion and Future work

In our work, we explored to test the influence brought by bilipschitz function towards the model performance in double pendulum experiment. Frankly speaking, there is a lot of future work that can be done in this field.

Firstly, the selection of bilipschitz functions is a big challenge. It is probably the most torturing part for the whole project. Even though we get theoretic proof for the functions designed according to Balan & Dock(2021), it doesn't work very well for this experiment. As a result, a more reasonable way to select the function or the function class with bilipschitz property is in need for future work. We may consider other classes of bilipschitz function to test whether it will improve the model performance.

Besides, the selection of vectors that get adjusted can be also taken into consideration. Currently, we replaced all vectors' inner product $\langle v_i, v_j \rangle$ with $\phi(v_i, v_j)$. However, it may achieve a better result if we selectively replace part of pairs, and leave the rest of them as the original inner product.

Lastly, issue of variable selection happens here. Given that we consider the scalar outputs of vectors q_1, p_1, q_2, p_2, the choice of scalar function and possible combination matter a lot for the final performance. In the original work(Yao et al., 2021), the researchers considered concatenating both the norm $||q_1 - p_1||$ and the square of it, $||q_1 - p_1||^2$, we may consider excluding one or both of them in future work. The dimension d of mapped vector for each sample, which is 30 in Yao's work, can also be tuned for better performance.

6 Acknowledgements

Thanks a lot for guidance of Prof Soledad Villar, Willson Gregory as my mentors for this project. Meanwhile, all precious related work done by previous researchers are indeed indispensable for my understanding for the whole project.

7 Reference

Balan, R., & Dock, C. B. (2022). Lipschitz analysis of generalized phase retrievable matrix frames. SIAM Journal on Matrix Analysis and Applications, 43(3), 1518-1571.

Chen, R. T., Rubanova, Y., Bettencourt, J., & Duvenaud, D. K. (2018). Neural ordinary differential equations. Advances in neural information processing systems, 31.

Greydanus, S., Dzamba, M., & Yosinski, J. (2019). Hamiltonian neural networks. Advances in neural information processing systems, 32.

Verine, A., Negrevergne, B., Chevaleyre, Y., & Rossi, F. (2023, April). On the expressivity of bi-Lipschitz normalizing flows. In Asian Conference on Machine Learning (pp. 1054-1069). PMLR.

Villar, S., Hogg, D. W., Storey-Fisher, K., Yao, W., & Blum-Smith, B. (2021). Scalars are universal: Equivariant machine learning, structured like classical physics. Advances in Neural Information Processing Systems, 34, 28848-28863.

Yao, W., Storey-Fisher, K., Hogg, D. W., & Villar, S. (2021). A simple equivariant machine learning method for dynamics based on scalars. arXiv preprint arXiv:2110.03761.

8 Appendix

8.1 Theory

8.1.1 Invariant and Equivariant function with scalars

In this setting, we are interested in learning a function that is invariant with respect to rotation. According to Villar et al.(2023), by First Fundamental Theorem for O(d), f is an O(d)-invariant scalar function with vectors $V = [v_1, v_2, \cdots, v_n]$ if and only if
$f(v_1, v_2, \cdots, v_n) = g(V^\top V) = g((v_i^\top v_j)_{1\leq i,j\leq n}^n)$, namely, $f(v_1, v_2, \cdots, v_n)$ can be written as a function of scalar products of v_i.

Meanwhile, by proposition, h is an O(d)-equivariant vector function of $V = [v_1, v_2, \cdots, v_n]$, if and only if there exists n O(d)-invariant scalar functions $f_t(\cdot)$ such that $h(v_1, v_2, \cdots, v_n) = \sum_{t=1}^{n} f_t(v_1, v_2, \cdots, v_n)v_t$

In this experiment, scalar results are computed for each vector pair with some function ψ returning a scalar. In the original work, $\psi_0(v_i, v_j)$ is the inner product $v_i^\top v_j$. In my work, a bilipschitz function ψ is designed to replace the output scalars.

8.1.2 Neural Ordinary Differential Equations

When we want to predict a sequence of states given the initial state, such as what we aim to achieve in this project(given $z^i(t_0)$, want to predict $z^i(t_j)$), we usually need to solve an ODE specified by some function ϕ: $\frac{d\mathbf{h}_t}{dt} = \phi_\theta(\mathbf{h}(t), t)$ (Chen et al., 2018). In order to approximate this function ϕ, we usually need to learn that from our data with a neural network, e.g., a multilayer perceptron.

8.1.3 Hamiltonian Neural Networks

Hamiltonian Neural Networks aims to give out those neural networks that not only fit the training data well, but also obey certain physics laws, such as mechanic energy conservation or Newton's Force Laws.(Greydanus et al., 2019)

By Hamiltonian Mechanics, given a set a coordinates $(\mathbf{q}, \mathbf{p})$, where $\mathbf{q} = (q_1, \cdots, q_N)$ denote the positions of the object set, and $\mathbf{p} = (p_1, \cdots, p_N)$ denote momentum of the object set(quite analogous to our double pendulum experiment setting). Define a scalar function $\mathcal{H}(p, q)$, and $\frac{d\mathbf{q}}{dt} = \frac{\partial \mathcal{H}}{\partial \mathbf{p}}$, $\frac{d\mathbf{p}}{dt} = -\frac{\partial \mathcal{H}}{\partial \mathbf{q}}$, so that $S_\mathcal{H} = (\frac{\partial \mathcal{H}}{\partial \mathbf{p}}, -\frac{\partial \mathcal{H}}{\partial \mathbf{q}})$ gives the time evolution of the system. We can construct ODEs according to $S_\mathcal{H}$, so that we can estimate later status of the system $z_j = (q_j, p_j)$ based on the previous status $z_{j-1} = (q_{j-1}, p_{j-1})$

8.1.4 Bilipschitz Function Background

A bijective function f(whose inverse always exists) is called a bilipschitz function if and only if both the function and its inverse have bilipschitz constants.(Verine et al.,2023)

Define the distance metric $D : R^{d\times r} \times R^{d\times r} \to R$ as the following,
$D(V_1, V_2) = \min_{U\in U(r)} ||V_1 - V_2U||$

Define a map $\theta : R^{d\times n} \to R^{d\times d}$, with $\theta(V) = (VV^\top)^{\frac{1}{2}}$. The square root of a symmetric matrix is defined via eigenvalue decomposition: $A = U\Lambda U^\top$, with U as an orthogonal matrix and Λ as a diagonal matrix, $A^{\frac{1}{2}} = U\Lambda^{\frac{1}{2}}U^\top$.

Definition: suppose $X, Y \in R^{n\times d}$, then X is equivalent to Y, namely, $X \sim Y$, $\Leftrightarrow \exists U \in O(d)$ such that $X = YU$

According to Balan & Dock(2021), suppose $r = \frac{||\theta(X)-\theta(Y)||_2}{D(X,Y)}$ with X is not equivalent to Y, we have $1 \le r \le \frac{1}{C_n}$, and $C_n = \begin{cases} 1, & d = 1 \\ \frac{\sqrt{2}}{2}, & d > 1 \end{cases}$, so that θ is a bilipschitz map, with the norm $||\cdot||_2$ for matrix as Frobenius norm.
Related experiments have been designed to verify this result. Code link: https://drive.google.com/file/d/1D-9D93D-H0ONc-gmyeHos3OoV2dCzOiM/view?usp=sharing

8.1.5 Bilipschitz Map

Given a number of vectors $V = [v_1, v_2, \cdots, v_n] \in R^{d\times n}$, suppose function $\psi : R^d \times R^d \to R$ is the ij-th element bilipschitz map, and $\psi_0 : R^d \times R^d \to R$ is the original map. Originally, the author utilized scalar method that $\psi_0(v_i, v_j) = (V^\top V)_{ij} = v_i^\top v_j$. After employing the bilipschitz map, it becomes: $\psi(v_i, v_j) = ((V^\top V)^{\frac{1}{2}})_{ij}$, by eigen decomposition, suppose $V^\top V = U\Sigma U^\top$, then we have $\psi(v_i, v_j) = (U\Sigma^{\frac{1}{2}}U^\top)_{ij}$

8.2 Previous Work

According to the work done by Yao et.al(2021), equivariant Neural Network model with scalar methods were employed to train the model, and it has demonstrated good performance.

8.2.1 Model Training Procedure

According to physical field knowledge, the model should obey equivariant property with respect to rotation and translation, which means the map of the rotation of one input vector should be equivalent to the rotation of the map of that input vector. Intuitively, we can rotate the initial speed direction and position arbitrarily, and finally we will arrive in the status where it only has the rotation compared to initial status for any given later stage. Thanks to important discoveries from Villar et.al(2021), a model is equivariant if and only if it employs inner product of vectors as scalars.

8.2.2 Methodology

The researcher utilized two approaches: Neural ordinary differential equations(Neural ODE), and Hamiltonian neural network(HNN).

Neural ODE:

Hamiltonian neural network: In this experiment, we parameterized the whole Neural ODE as:
$\frac{d\mathbf{z}}{dt} = F(\mathbf{z}(t), t) = F((\mathbf{q}_1, \mathbf{p}_1, \mathbf{q}_2, \mathbf{p}_2), \mathbf{q}_o, g)$

By First Fundamental Theorem for O(d), we have
$F((\mathbf{q}_1, \mathbf{p}_1, \mathbf{q}_2, \mathbf{p}_2), \mathbf{q}_o, g) = \sum_{u\in\mathcal{I}} \mathbf{g}_u(\{\sigma(e^\top e') : e, e' \in \mathcal{I}, \sigma \in \Omega\})u$, as an equivariant model with respect to rotation and translation,
where $\mathcal{I} = \{\mathbf{q}_1 - \mathbf{q}_o, \mathbf{q}_1 - \mathbf{q}_o, \mathbf{q}_1 - \mathbf{q}_o, \mathbf{p}_1, \mathbf{p}_2, \mathbf{g}\}$, $\Omega = \{\sigma_1 : x \mapsto x, \sigma_2 : x \mapsto \sqrt{|x|}\}$.

Finally, given $\hat{z}(t_{j-1})$, and $\hat{z}(t_0) = z(t_0)$, we can iteratively compute the whole rollout trajectory $(z_0, \hat{z}(t_1), \cdots, \hat{z}(t_T)$
Specifically, given $\hat{z}(t_{j-1}), t_{j-1}, t_j, F$, and the equation $\frac{d\mathbf{z}}{dt} = F(\mathbf{z}, t)$, we can derive $\hat{z}(t_j)$ by some ODE solver.

HNN:

	Scalars O(3)	EMLP				MLP
		O(2)	SO(2)	D_2	D_6	
N-ODEs:	$\mathbf{.009 \pm .001}$	.020±.002	.051±.036	.023±.002	.036±.025	.048±.000
HNNs:	$\mathbf{.005 \pm .002}$	.012±.002	.016±.003	.111±.167	.013±.002	.028±.001

Figure 2: Springy Double Pendulum

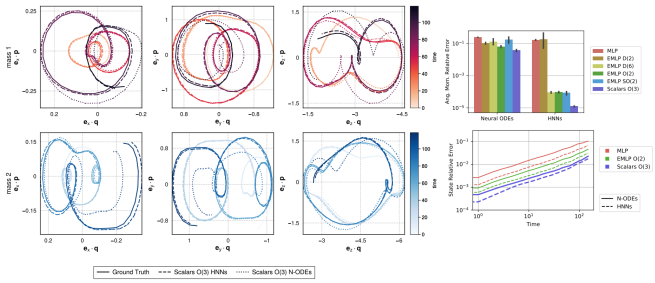

Figure 3: Springy Double Pendulum

In double pendulum experiment, the Hamiltonian function $\mathcal{H}$ was modeled as:
$\mathcal{H}(q_1, p_1, q_2, p_2, q_o, g) = h(\{\sigma(e^\top e') : e, e' \in \mathcal{I}, \sigma \in \Omega\})$, where $\mathcal{I} = \{\mathbf{q}_1 - \mathbf{q}_o, \mathbf{q}_1 - \mathbf{q}_o, \mathbf{q}_1 - \mathbf{q}_o, \mathbf{p}_1, \mathbf{p}_2, \mathbf{g}\}$ as the same notion of neural ODE, and h is a scalar function based on inner products in $\mathcal{Q}$

Ultimately, we derive: $\frac{d\mathbf{z}(t)}{dt} = \mathbf{J}\nabla\mathcal{H}$, where $\mathbf{J} = [\mathbf{0}, \mathbf{I}; -\mathbf{I}, \mathbf{0}]$ after concatenating q and p. Then according to the ODE, we can compute the later status $z(t_j)$ based on the previous status z_{j-1} by an ODE solver.

8.2.3 Comparison with baseline model

As what we can see in the error summary table below in Figure 2,(Yao et al., 2021), compared with common equivariant machine learning models and conventional MLP models, scalar method demonstrated much better model performance in terms of the lower error rate.

8.2.4 Trajectory Prediction Visualization

Meanwhile, the author also generated the visualization plot of trajectory prediction in Figure 3. As we can see, scalar HNN method has shown very satisfactory performance since it is so close to the ground truth trajectory visually.

8.3 Methodology

We hope to train a machine learning model which plays a same role as the original setting, but replace the inner product with the function value where the function f has bilipschitz property. Recall that the function was proposed as $\psi : R^d \times R^d \to R$, and $\psi(v_i, v_j) = ||(v_i v_j^\top)^{\frac{1}{2}}||_2$. Given $\mathbf{Z}_{train} \in R^{N \times 12}$, we have N samples.

For each sample j, the element $\mathbf{Z}^j_{train} = \begin{bmatrix} q_1^j \\ p_1^j \\ q_2^j \\ p_2^j \end{bmatrix} \in R^{4\times 3}$, then we compute $M^j = Z^j_{train}(Z^j_{train})^\top \in R^{4\times 4}$,

and $m_1^j = (M^j)^{\frac{1}{2}} \in R^{16}$ after flattening the matrix into a vector. Besides, compute the norm of each of $q_1^j, p_1^j, q_2^j, p_2^j$, then get $m_2^j = \begin{bmatrix} ||q_1^j||_2 \\ ||p_1^j||_2 \\ ||q_2^j||_2 \\ ||p_2^j||_2 \end{bmatrix} \in R^4$ where L_2 norm is applied. Finally, concatenate m_1 and m_2 so that we get $m^j = \begin{bmatrix} m_1^j \\ m_2^j \end{bmatrix} \in R^{20}$

We compute the inner product of g and z^j for each of four elements in z^j, so that we get $s_1^j = \begin{bmatrix} \langle g, q_1^j \rangle \\ \langle g, p_1^j \rangle \\ \langle g, q_2^j \rangle \\ \langle g, p_2^j \rangle \end{bmatrix} \in R^4$,

Then, compute $s_2^j = \begin{bmatrix} ||q_1^j - p_1^j||^2 \\ ||q_1^j - p_1^j|| \end{bmatrix} \in R^2$

And compute $s_3^j = \begin{bmatrix} \langle q_1^j - p_1^j, q_1^j \rangle \\ \langle q_1^j - p_1^j, p_1^j \rangle \\ \langle q_1^j - p_1^j, q_2^j \rangle \\ \langle q_1^j - p_1^j, p_2^j \rangle \end{bmatrix} \in R^4$,

Lastly, concatenate previous vectors to get $k^j = \begin{bmatrix} m^j \\ s_1^j \\ s_2^j \\ s_3^j \end{bmatrix} \in R^{30}$,

Repeat the same process for each sample, finally we get $K = \begin{bmatrix} k^1 \\ k^2 \\ \vdots \\ k^N \end{bmatrix} \in R^{N \times 30}$.

Then we use the matrix K as the input training data into the neural network.